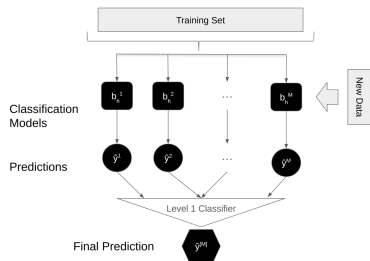


# Applied Machine Learning

## Ensembling: Stacking

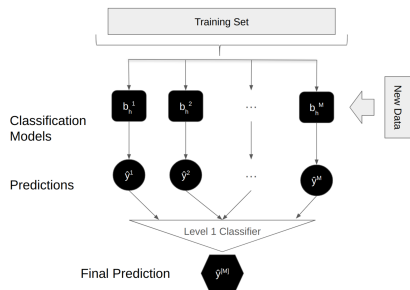


### Learning goals

- What is Stacking?
- How can we stack multiple layers?

# STACKING

Also called stacked generalization (► WOLPERT1992): train multiple base models (level 0) on the same data and use their predictions as input features for a level 1 model that makes the final ensemble prediction → heterogeneous ensembling



Steps:

- 1 Base model training (level 0)
- 2 Collect predictions
- 3 level 1 model training

In contrast to model averaging, stacking already combines predictions during training.

# STACKING

**Algorithm:** Single Layer Stacking

**Require:** Training data  $\mathcal{D} = \left( \left( \mathbf{x}^{(1)}, y^{(1)} \right), \dots, \left( \mathbf{x}^{(n)}, y^{(n)} \right) \right)$

**Ensure:** Stacked Ensemble  $f^{[M]}(\mathbf{x})$

**Step 1: Learn level 0 base models**

**for**  $m \leftarrow 1$  to  $M$  **do**

    Fit base model  $b^{[m]}$  on  $\mathcal{D}$

**end for**

**Step 2: Construct new dataset  $\mathcal{D}'$  from  $\mathcal{D}$**

**for**  $i \leftarrow 1$  to  $n$  **do**

    Add  $(\mathbf{x}'^{(i)}, y^{(i)})$  to new dataset, where  $\mathbf{x}'^{(i)} = (b^{[1]}(\mathbf{x}^{(i)}), \dots, b^{[M]}(\mathbf{x}^{(i)}))$

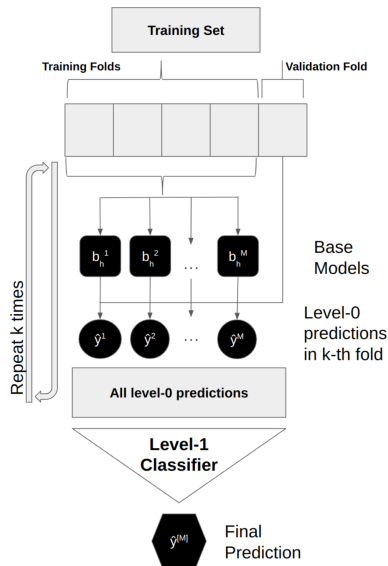
**end for**

**Step 3: Fit level 1 model  $f^{[M]}(\mathbf{x})$  on  $\mathcal{D}'$**

**return**  $f^{[M]}(\mathbf{x})$

NB: We can either drop the original features or keep them when learning the level 1 model.

# STACKING CROSS-VALIDATED



Using cross-validated predictions of the base models

- improve generalization
- reduce overfitting
- enhances base model diversity
- minimizes information leakage

NB: Stacking with a simple hold-out split instead of cross-validation is known as *blending*.

# STACKING CROSS-VALIDATED

**Algorithm:** Single Layer Stacking with Cross-Validation

**Require:** Training data  $\mathcal{D} = \left( \left( \mathbf{x}^{(1)}, y^{(1)} \right), \dots, \left( \mathbf{x}^{(n)}, y^{(n)} \right) \right)$ , number of folds  $K$

**Ensure:** Stacked Ensemble  $f^{[M]}(\mathbf{x})$

for  $k \leftarrow 1$  to  $K$  do

**Step 1: Learn level 0 base models**

    for  $m \leftarrow 1$  to  $M$  do

        Fit base model  $b^{[m]}$  on  $\mathcal{D}_{\text{train}}^{(k)}$

    end for

**Step 2: Add out-of-fold predictions to new dataset  $\mathcal{D}'$**

    for all  $(\mathbf{x}^{(i)}, y^{(i)}) \in \mathcal{D}_{\text{test}}^{(k)}$  do

        Add  $(\mathbf{x}'^{(i)}, y^{(i)})$  to new dataset, where  $\mathbf{x}'^{(i)} = (b^{[1]}(\mathbf{x}^{(i)}), \dots, b^{[M]}(\mathbf{x}^{(i)}))$

    end for

end for

**Step 3: Fit level 1 model  $f^{[M]}(\mathbf{x})$  on  $\mathcal{D}'$**

return  $f^{[M]}(\mathbf{x})$

# MULTILEVEL STACKING

So far: train level 0 base models, use predictions to train the level 1 model.

Now: Also train multiple models at level 1, use predictions to train final level 2 model

- 1 level 0 base model training
- 2 Collect predictions of level 0 base models
- 3 level 1 model training
- 4 Collect predictions of level 1 models
- 5 level 2 final model training

NB: In principle we can repeat this training of multiple models at the  $l$ -th level indefinitely creating deep stacked ensembles.

# HIGHER LEVEL MODELS

Popular choices of level 1 or level 2 models in stacking include:

- linear / logistic regression
- GES (average the predictions of the models at the previous level)

Higher-level training data:

- Probabilities instead of class labels (see [▶ TINGWITTEN1999](#))
- Include original dataset **X** or not?

# FORWARD PASS / PREDICTION

**Assume:** trained base models  $b^{[1]}, \dots, b^{[M]}$  and meta-model  $f^{[M]}(\mathbf{x})$

**Require:** Test data  $\mathbf{X}_{\text{test}} = (\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(n_{\text{test}})})$

**Ensure:** Predictions  $\hat{\mathbf{y}}_{\text{test}}$

**Step 1: Predict with each base model**

**for**  $m \leftarrow 1$  to  $M$  **do**

$\hat{\mathbf{y}}^{[m]} \leftarrow b^{[m]}(\mathbf{X}_{\text{test}})$

**end for**

**Step 2: Construct meta-feature matrix**

$\mathbf{X}' \leftarrow [\hat{\mathbf{y}}^{[1]}, \dots, \hat{\mathbf{y}}^{[M]}]$  or  $\mathbf{X}' \leftarrow [\mathbf{X}_{\text{test}}, \hat{\mathbf{y}}^{[1]}, \dots, \hat{\mathbf{y}}^{[M]}]$  (if original features are retained)

**Step 3: Predict with the meta-model**

**return**  $f^{[M]}(\mathbf{X}')$



# TWO QUESTIONS

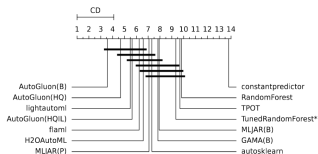
What other methods do the following level-1 classifiers recover?

- Fit the level-1 training data with a linear model that only depends on one feature, and that feature is chosen such that  $\rho$ , our metrics of interest, is optimized.
- The level-1 model is a linear model that assign equal weight  $w_m = \frac{1}{M}$  to every input feature.

# AUTOGLUON



<https://auto.gluon.ai/>



**Figure:** AutoGluon Performance on AMLB 1h binary classification.

- The de-facto most popular and successful AutoML system
- Developed and maintained by Amazon
- Can also work with multimodal and time-series data
- Does not use HPO to find well performing models
  - Instead, uses a set of default configurations for different learners
- Employs multi-layer stacking using the same set of default models in higher layers as for the base layer
- Uses greedy ensemble selection to combine the predictions of the last layer